

Relatório de Evolução - Sprint 04

ChargeOps Assistant - EV Challenge FIAP e GoodWe - Grupo 6 - Turma 1CCR - Entrega 23/10/2026

1. Resumo da evolução

Nas Sprints 1 e 2 o chatbot respondia com dados fixos escritos direto no código, sem buscar nada em nenhum lugar: tudo que ele sabia estava hardcoded. Na Sprint 03 o projeto virou uma chain em LangChain, com memória de conversa e guardrails, mas ainda sem RAG, os dados continuavam fixos no código. Na Sprint 04 o projeto ganhou RAG de verdade: os documentos do condomínio (manual do carregador, regimento, tabela de tarifas e FAQ) foram colocados em PDF, divididos em pedaços, transformados em vetores com o nomic-embed-text e guardados num banco Chroma persistente. Agora a resposta vem da busca nesses documentos, com a fonte citada, e tem uma interface web feita em Gradio para conversar com o assistente.

2. Pipeline RAG

A base de conhecimento tem 4 PDFs: o manual do carregador ChargeGrid, o regimento de recarga compartilhada, a tabela de tarifas e taxas e as perguntas frequentes. Os PDFs são lidos com PyMuPDFLoader e divididos em pedaços com RecursiveCharacterTextSplitter.

A primeira tentativa de divisão usava pedaços pequenos (chunk_size 300) e sem sobreposição, o que fragmentava demais as seções e às vezes cortava uma frase ou um valor no meio. A segunda tentativa usa pedaços maiores (chunk_size 900) com sobreposição de 150 caracteres, o que mantém cada seção mais inteira. O k (quantidade de pedaços buscados por pergunta) também subiu de 2 para 4, trazendo mais contexto.

O prompt também mudou entre as duas versões: a primeira só pedia para responder com base no contexto, sem regra de citação nem de recusa. A segunda exige citar a fonte em todo texto, recusar pergunta fora do contexto, nunca inventar número e tratar o conteúdo recuperado como dado, nunca como instrução, mesmo que o documento tenha algo parecido com um comando escondido.

3. Comparativo antes e depois

Métrica	Sprints 1/2 (versão original)	Sprint 04 (RAG avaliado)
Busca semântica / RAG	Não existia, dados fixos no código	Sim, ChromaDB + nomic-embed-text
Nota média no eval (0 a 10)	Não aplicável, sem eval de RAG	8,18 (iteração 2)
Faithfulness médio (RAGAS ou fallback)	Não aplicável	1,00
Answer relevancy médio (RAGAS ou fallback)	Não aplicável	0,87
Citação de fonte nas respostas	Nunca citava fonte	89% das respostas
Qualidade do contexto recuperado	Não buscava contexto nenhum	4 trechos por pergunta, chunk de 900 caracteres
Recusa de pergunta fora do contexto	Não tinha essa regra	Sim, por limiar de relevância antes de chamar o modelo
Proteção contra instrução escondida em documento	Não existia	Sim, filtro de linha suspeita antes de montar o contexto
Interface	Só terminal	Interface web em Gradio

Scores por iteração

Iteração	chunk_size / overlap	k	Nota eval (0 a 10)	Faithfulness	Answer relevancy	Com citação
1 (ingênua)	300 / 0	2	3,94	1,00	0,88	0%
2 (grounding + citação)	900 / 150	4	8,18	1,00	0,87	89%

Mais detalhe sobre cada caso de teste e sobre como as métricas foram calculadas está em docs/relatorio_rag.md. A comparação entre modelos de geração (temperature, top_p, max_tokens e k documentados) está em docs/relatorio_modelos.md.

4. Problemas encontrados e soluções

Problema	Causa	Solução
O RAGAS não importava (ModuleNotFoundError).	A versão instalada do ragas tenta carregar langchain_community.chat_models.vertexai.ChatVertexAI, que não existe mais nessa versão do langchain-community. Aconteceu com a 0.4.3 e com a 0.2.15.	Usamos o fallback manual documentado que o próprio enunciado permite: faithfulness checando números e códigos contra o contexto recuperado, e answer relevancy por similaridade de cosseno entre pergunta e resposta.
Pedaços pequenos (iteração 1) fragmentavam demais as seções.	Chunk de 300 caracteres sem sobreposição corta uma tabela ou uma frase bem no meio, perdendo parte da informação.	Aumentar o chunk para 900 caracteres com 150 de sobreposição, mantendo cada seção mais inteira dentro de um único pedaço.
Um documento poderia conter um texto parecido com uma instrução escondida.	Se alguém colocasse algo como 'ignore as instruções anteriores' dentro de um PDF da base, o modelo poderia tentar seguir aquilo como se fosse um comando.	Um filtro roda antes de montar o contexto: troca só a linha suspeita por um aviso, sem apagar o resto do trecho, e o prompt também reforça que o conteúdo do contexto é dado, nunca instrução.

5. Equipe

Integrante	RM
João Lucas	571355
Filipe Gunther	571131
Guilherme Guimaraes	572957
Enzo de Freitas	572037
David Gabriel	574147
Lucas Pinheiro	573497